Décodage d'appels vocaux à partir d'un fichier PCAPNG à l'aide de `tshark` et `ffmpeg`

Ce guide explique la méthodologie étape par étape utilisée pour extraire et décoder des flux audio VoIP à partir du fichier de capture de paquets `capture (1).pcapng`.

Étape 1 : Découverte et analyse des protocoles

Les captures VoIP standard contiennent généralement des paquets SIP (Session Initiation Protocol) qui négocient le codec vocal et les ports UDP via le protocole SDP (Session Description Protocol). En l'absence de paquets SIP dans la capture, Wireshark et `tshark` ne savent pas automatiquement quels paquets UDP contiennent de l'audio, et les traitent à la place comme des données UDP brutes.

1. **Vérifier la signalisation SIP :**
   L'exécution d'un filtre pour SIP n'a donné aucun résultat :
   ```
   tshark -r "capture (1).pcapng" -Y "sip"
   ```
2. **Analyser les conversations UDP :**
   Pour localiser les flux audio, nous avons recherché les conversations UDP à haut volume entre les hôtes :

   ```
   tshark -r "capture (1).pcapng" -q -z conv,udp
   ```

   Cela a révélé deux conversations UDP locales actives majeures :
   - **Appel 1 (actif d'environ 17s à 43s) :** entre les ports `54700` et `62398`.
   - **Appel 2 (actif d'environ 70s à 106s) :** entre les ports `55512` et `50574`.
3. **Forcer le décodage en tant que RTP :**
   Nous avons demandé à `tshark` de décoder le trafic sur ces ports en tant que RTP :

   ```
   tshark -r "capture (1).pcapng" \
     -d udp.port==54700,rtp -d udp.port==62398,rtp \
     -d udp.port==55512,rtp -d udp.port==50574,rtp \
     -q -z rtp,streams
   ```

   Cette opération a permis d'identifier avec succès quatre flux vocaux actifs ayant le type de charge utile (payload) dynamique **96** (SSRC : `0x0B478DAE`, `0xD21A9A78`, `0x62F226EF` et

`0x8D90AB7A`).

Étape 2 : Identification du codec

Pour identifier le codec utilisé, nous avons examiné les champs d'en-tête RTP :

```
tshark -r "capture (1).pcapng" \
  -d udp.port==54700,rtp -d udp.port==62398,rtp \
  -Y "rtp" -T fields -e rtp.seq -e rtp.timestamp -e rtp.payload
```

- **Fréquence d'échantillonnage :** Les horodatages (timestamps) RTP augmentaient de très exactement `960` unités par paquet. Étant donné que l'intervalle entre les paquets (delta) était de `20 ms` (50 paquets par seconde), la fréquence d'échantillonnage a été calculée comme suit :

$$\text{éence décantllonnae} = \frac{960 \text{ écantllons}}{0,02 \text{ seconde}} = 48\,000 \text{ Hz}$$

- **Taille de la charge utile (payload) :** La taille de la charge utile variait paquet par paquet (par ex. 28 octets, 45 octets, 37 octets).
- **Conclusion :** Un codec à débit binaire variable (VBR) fonctionnant à 48 kHz est la signature caractéristique du codec audio **Opus**.

Étape 3 : Décodage des flux audio

L'interface en ligne de commande de `tshark` ne transcodant pas nativement les paquets RTP Opus en fichiers audio lisibles, nous avons écrit un script Python pour automatiser le processus d'extraction et de décodage.

La technique de décodage :

1. **Extraction des paquets :** Le script a utilisé `tshark` pour extraire la charge utile UDP complète (qui comprend l'en-tête RTP de 12 octets + les données de charge utile Opus) pour un SSRC spécifique :

   ```
   tshark -r "capture (1).pcapng" -d udp.port==54700,rtp -Y "rtp.ssrc ==
   0x0B478DAE" -T fields -e udp.payload
   ```

2. **Modèle SDP :** Nous avons créé un fichier temporaire de protocole de description de session (`.sdp`) pour indiquer à `ffmpeg` comment interpréter le flux :

```
v=0
o=- 0 0 IN IP4 127.0.0.1
s=No Name
c=IN IP4 127.0.0.1
t=0 0
m=audio 12345 RTP/AVP 96
a=rtpmap:96 opus/48000/2
```

3. **Transmission via socket UDP :**
   - Nous avons lancé `ffmpeg` en écoute sur le port local `12345` avec le fichier SDP comme entrée.
   - Le script Python a ouvert un socket UDP et a transmis les paquets RTP bruts vers `127.0.0.1:12345`.
   - La transmission a été cadencée à un intervalle de `10 ms` pour s'assurer que le tampon UDP ne déborde pas et ne perde pas de trames.
   - `ffmpeg` a analysé les en-têtes RTP, a décodé la charge utile Opus et a écrit l'audio dans un fichier WAV standard.

Cette opération a été répétée pour les quatre SSRC :

- `call1_stream1.wav` (SSRC `0x0B478DAE`)
- `call1_stream2.wav` (SSRC `0xD21A9A78`)
- `call2_stream1.wav` (SSRC `0x62F226EF`)
- `call2_stream2.wav` (SSRC `0x8D90AB7A`)

Étape 4 : Fusion des appels

Les connexions VoIP capturent les voix entrantes et sortantes dans deux flux unidirectionnels distincts. Pour reconstruire les appels complets, nous avons fusionné les flux correspondants à l'aide du filtre `amix` de `ffmpeg` :

```
# Mélanger l'Appel 1
ffmpeg -i call1_stream1.wav -i call1_stream2.wav -filter_complex
amix=inputs=2:duration=longest call1_mixed.wav

# Mélanger l'Appel 2
ffmpeg -i call2_stream1.wav -i call2_stream2.wav -filter_complex
amix=inputs=2:duration=longest call2_mixed.wav
```

Fichiers de sortie finaux générés (dans `/home/monga/Downloads/`) :

- **call1_mixed.wav** (~26 secondes d'audio)
- **call2_mixed.wav** (~49 secondes d'audio)